Backend Engineering Interview Assignment

Multi-Courier Integration Platform

1. Overview

You are building the backend service for an e-commerce logistics platform. Today, the company integrates with one courier partner — UrbaneBolt. In the coming months, the company plans to onboard several more courier partners (e.g., Delhivery, Shiprocket, Bluedart, DTDC).

Your task is to design and implement a courier integration system in which:

- UrbaneBolt is the first concrete integration.
- Adding any new courier partner in the future requires minimal code changes
- Our API consumers (e.g., the order management system, web frontend) call ONE unified API. They only pass a courier_partner identifier;

2. Reference API

Use the UrbaneBolt UAT API as the first courier integration:

API Documentation: https://bit.ly/ease-commerce-assignment

Cover at minimum the following operations: authentication, create order / shipment, track shipment / get status, and cancel order. If the docs expose more operations, you may pick the relevant subset.

3. Functional Requirements

3.1 Unified API for Consumers

Expose your own REST API endpoints to internal consumers. The contract must be courier-agnostic. Examples:

```
POST /api/v1/orders
GET  /api/v1/orders/{order_id}/track
POST /api/v1/orders/{order_id}/cancel
POST /api/v1/orders/bulk
```

Every request body must accept a courier_partner field (e.g., "urbanebolt", "delhivery"). The rest of the payload is YOUR internal, normalized schema. The caller does NOT know or care what UrbaneBolt's payload looks like.

3.2 Pluggable Courier Architecture

Adding a new courier partner tomorrow MUST NOT require:

- Changing controllers / routes.
- Changing the unified request/response DTOs.
- Modifying existing courier implementations.
- Modifying business logic in services.

3.3 Order & Tracking Persistence

Persist every order in your database with at least the following information:

- Internal order ID.
- Courier partner used.
- Courier's order/shipment ID returned by the partner.
- AWB number (Air Waybill / tracking number) returned by the courier.
- Current shipment status (e.g., CREATED, PICKED_UP, IN_TRANSIT, DELIVERED, CANCELLED, FAILED).
- Full request payload sent to courier and full response received (for audit/debugging).
- Created/updated timestamps.

Tracking history must be stored as a separate, append-only table — every status update from the courier should be recorded with timestamp and raw payload.

3.4 Bulk Order Processing (100 Orders)

Provide a bulk-create endpoint that accepts up to 100 orders in a single request:

```
POST /api/v1/orders/bulk
```

Requirements:

- Process orders concurrently / asynchronously — do NOT call the courier 100 times sequentially in one HTTP request.
- Each order in the batch may use a different courier_partner.
- Partial success must be handled: if 95 succeed and 5 fail, the response must clearly indicate per-order success/failure with reason.
- The endpoint must be idempotent on the order_id field — submitting the same order_id twice must not create two shipments.
- The system should remain responsive — consider returning a batch_id immediately and processing in the background, or streaming results. Document your choice and trade-offs.

3.5 Error Handling

Errors must be handled cleanly at every layer:

- Validation errors (bad input) → HTTP 400 with a clear field-level error structure.
- Unknown courier_partner → HTTP 400 with a list of supported couriers.

- Courier API errors (4xx from courier) → mapped to your own normalized error codes; do NOT leak the courier's raw error to the client.
- Courier API failures (5xx, timeout, network) → retry with backoff (configurable), then fail gracefully and persist the failure for later reconciliation.
- Authentication failures with the courier (e.g., expired token) → automatic re-authentication and one retry.
- Every failure must be logged with: order_id, courier_partner, request id, error type, and stack trace where appropriate.

Define a single, normalized error response shape used by ALL your endpoints:

4. Non-Functional Requirements

- Configuration-driven: API keys, base URLs, timeouts, retry counts must come from config / env, never hardcoded.
- Documentation: a README explaining setup, design choices, how to add a new courier, and assumptions.

5. Tech Stack

You may choose your stack, but we expect production-quality code.

6. Deliverables

1. Public Git repository (GitHub/GitLab) with the full source code.
2. README.md with: setup steps, environment variables, how to run, how to test, and how to add a new courier.
3. A short DESIGN.md (1–2 pages) describing your architecture, the design pattern used (and why), database schema, and trade-offs.
4. Postman collection or curl examples for all your endpoints.
5. (Bonus) A second mock courier adapter (e.g., MockCourier) demonstrating that your design truly supports plug-in couriers.

— End of Assignment —